INTRODUCTION TO NORMALIZATION

When developing the schema of a relational database, one of the most important aspects to be taken into account is to ensure that the duplication is minimized. This is done for two purposes:

- Reducing the amount of storage needed to store the data.
- Avoiding unnecessary data conflicts that may creep in because of multiple copies of the same data getting stored.

Normalization is the process of minimizing redundancy from a relation or set of relations. Redundancy in relation may cause insertion, deletion and update anomalies. So, it helps to minimize the redundancy in relations. Normal forms are used to eliminate or reduce redundancy in database tables. It is a database design technique which organizes tables in a manner that reduces redundancy and dependency of data. It divides larger tables to smaller tables and links them using relationships.

PROBLEMS WITHOUT NORMALIZATION

If a table is not properly normalized and has data redundancy then it will not only eat up extra memory space but will also make it difficult to handle and update the database, without facing data loss. Insertion, updating and Deletion Anomalies are very frequent if database is not normalized. To understand these anomalies let us take an example of a Student table.

Student

Sid	Sname	Branch	HOD	Office Phone
100	Abin	Pokhara branch	Mr. Nabin	4434564
101	Aarav	Pokhara branch	Mr. Nabin	4434564
102	Ashana	Pokhara branch	Mr. Nabin	4434564
103	Geeta	Pokhara branch	Mr. Nabin	4434564
104	Umesh	Banepa branch	Mr. Prabin	5212343
105	Ramesh	Banepa branch	Mr. Prabin	5212343

In the table above, we have data of 6 Computer Sci. students. As we can see, data for the fields branch, HOD and Office_Phone is repeated for the students who are in the same branch in the college, this is Data Redundancy.

Insertion Anomaly

This occurs when an attempt to insert data into a non-existent record. Suppose for a new admission, until and unless a student opts for a branch, data of the student cannot be inserted, or else we will have to set the branch information as NULL.

Also, if we have to insert data of 100 students of same branch, then the branch information will be repeated for all those 100 students. These scenarios are nothing but Insertion anomalies.

Update Anomaly

When several instances of the same data are scattered across the database without proper relationship/link, it could cause strange conditions where a few of the instances will get updated with new values whereas some of them will not. This leaves the database in an inconsistent state.

What if Mr. Prabin leaves the college? Or is no longer the HOD of computer science department? In that case all the student records will have to be updated, and if by mistake we miss any record, it will lead to data inconsistency. This is Updating anomaly.

Deletion Anomaly

In our Student table, two different information are kept together, Student information and Branch information. Hence, at the end of the academic year, if student records are deleted, we will also lose the branch information. This is Deletion anomaly.

WHY WE NEED NORMALIZATION?

Normalization is the aim of well design Relational Database Management System (RDBMS). It is step by step set of rules by which data is put in its simplest forms. We normalize the relational database management system because of the following reasons:

- Minimize data redundancy i.e. no unnecessarily duplication of data.
- To make database structure flexible i.e. it should be possible to add new data values and rows without reorganizing the database structure.
- Data should be consistent throughout the database i.e. it should not suffer from following anomalies.
- **Insert Anomaly** - Due to lack of data i.e., all the data available for insertion such that null values in keys should be avoided. This kind of anomaly can seriously damage a database
- **Update Anomaly** - It is due to data redundancy i.e. multiple occurrences of same values in a column. This can lead to inefficiency.
- **Deletion Anomaly** - It leads to loss of data for rows that are not stored elsewhere. It could result in loss of vital data.
- Complex queries required by the user should be easy to handle.

Advantages of Normalization

The following are the advantages of the normalization.

- More efficient data structure.
- Avoid redundant fields or columns.
- More flexible data structure i.e. we should be able to add new rows and data values easily
- Better understanding of data.
- Ensures that distinct tables exist when necessary.
- Easier to maintain data structure i.e. it is easy to perform operations and complex queries can be easily handled.
- Minimizes data duplication.
- Close modeling of real world entities, processes and their relationships.
- A smaller database can be maintained as normalization eliminates the duplicate data. Overall size of the database is reduced as a result.

- Better performance is ensured which can be linked to the above point. As databases become lesser in size, the passes through the data becomes faster and shorter thereby improving response time and speed.

Disadvantages of Normalization

The following are disadvantages of normalization.

- More tables to join as by spreading out data into more tables, the need to join table's increases and the task becomes more tedious. The database becomes harder to realize as well.
- Tables will contain codes rather than real data as the repeated data will be stored as lines of codes rather than the true data. Therefore, there is always a need to go to the lookup table.
- Data model becomes extremely difficult to query against as the data model is optimized for applications, not for ad hoc querying. (Ad hoc query is a query that cannot be determined before the issuance of the query. It consists of an SQL that is constructed dynamically and is usually constructed by desktop friendly query tools.). Hence it is hard to model the database without knowing what the customer desires.
- As the normal form type progresses, the performance becomes slower.
- It is very time consuming and difficult process in normalizing relations of higher degree.
- Careless decomposition may leads to bad design of database which may leads to serious problems.
- Proper knowledge is required on the various normal forms to execute the normalization process efficiently. Careless use may lead to terrible design filled with major anomalies and data inconsistency.

FUNCTIONAL DEPENDENCIES

The functional dependency is a relationship that exists between two attributes. It typically exists between the primary key and non-key attribute within a table.

If R is a relation with attributes X and Y, a functional dependency between the attributes is represented as X→Y, which specifies Y is functionally dependent on X. Here X is a determinant set and Y is a dependent attribute. Each value of X is associated with precisely one Y value.

Functional dependency in a database serves as a constraint between two sets of attributes. Defining functional dependency is an important part of relational database design and contributes to aspect normalization.

Example 4.1: Suppose we have a student table with attributes: Stu_Id, Stu_Name, Stu_Age. Here Stu_Id attribute uniquely identifies the Stu_Name attribute of student table because if we know the student id we can tell the student name associated with it. This is known as functional dependency and can be written as Stu_Id→Stu_Name or in words we can say Stu_Name is functionally dependent on Stu_Id.

TYPES OF FUNCTIONAL DEPENDENCY

There are many types of functional dependencies, depending on several criteria

- Trivial and Non-trivial dependencies
- Fully functional dependency
- Partial functional dependency
- Transitive dependency
- Multivalued dependency

Trivial and Non trivial functional dependency

The dependency of an attribute on a set of attributes is known as trivial functional dependency if the set of attributes includes that attribute. Symbolically: A ->B is trivial functional dependency if B is a subset of A. The following dependencies are also trivial:

- A→A
- B→B

Example 4.2: Consider a table with two columns Student_id and Student_Name. {Student_Id, Student_Name}→Student_Id is a trivial functional dependency as Student_Id is a subset of {Student_Id, Student_Name}. That makes sense because if we know the values of Student_Id and Student_Name then the value of Student_Id can be uniquely determined. Also, Student_Id→Student_Id & Student_Name→Student_Name are trivial dependencies too.

Non trivial functional dependency

If a functional dependency X→Y holds true where Y is not a subset of X then this dependency is called non trivial Functional dependency

Example 4.3: Consider an employee table with three attributes: {emp_id, emp_name, emp_address}

The following functional dependencies are non-trivial:

emp_id → emp_name (emp_name is not a subset of emp_id)

emp_id → emp_address (emp_address is not a subset of emp_id)

Fully Functional Dependency

An attribute is fully functionally dependent on a second attribute if and only if it is functionally dependent on the second attribute but not on any subset of the second attribute. Mathematically, for a relation schema R, in a functional dependency X→Y, Y is said to be fully functional dependent on X if Z→Y is false for all $Z \subset X$.

Partial Functional Dependency

An attribute is partial functionally dependent on a second attribute if and only if it is functionally dependent on the second attribute and also dependency occur on any subset of the second attribute. This is the situation that exists if it is necessary to only use a subset of the attributes of the composite determinant to identify its object. Mathematically, for a relation schema R, in a functional

dependency X→Y, Y is said to be partial functional dependent on X if by removal of some attributes from X and the dependency still holds.

Example 4.4: The dependency {Emp-Id, Projecta-No} → {Emp-Name} is partial because Emp-Id→Emp-Name also holds.

Transitive dependency

A functional dependency is said to be transitive if it is indirectly formed by two functional dependencies. For example, X →Z is a transitive dependency if the following three functional dependencies hold true:

- X→Y
- Y→Z

⇨ X→Z

Example 4.5: Let's take an example to understand it better

Movie_ID	Listing_ID	Listing_Type	DVD_Price ($)
M08	L09	Crime	180
M03	L05	Drama	250
M05	L09	Crime	180

Movie_ID →Listing_ID

Listing_ID →Listing_Type

Also, Movie_ID →Listing_Type

Multivalued dependency

Multivalued dependency occurs when there are more than one independent multivalued attributes in a table.

Example 4.6: Consider a bike manufacture company, which produces two colors (Black and white) in each model every year.

Bike_model	Manuf_year	Color
M1001	2007	Black
M1001	2007	Red
M2012	2008	Black
M2012	2008	Red
M2222	2009	Black
M2222	2009	Red

Here columns manuf_year and color are independent of each other and dependent on bike_model. In this case these two columns are said to be multivalued dependent on bike_model. These dependencies can be represented like this:

bike_model →→ manuf_year

This can be read as "bike_model multidetermined manful_year".

TYPES OF NORMALIZATION

The degree of normalization is defined by normal forms. The normal forms in an increasing level of normalization are first normal form (1NF), second normal form (2NF), 3NF, Boyce-Codd normal form, 4NF and 5NF. Each normal form is a set of conditions on a schema that guarantees certain properties relating to redundancy and update anomalies. In general 3NF is considered good enough. In certain instances, a lower level of normalization, that is the instance where queries take enormous time to execute.

There are various database "Normal" forms. Each normal form has an importance which helps in optimizing the database to save storage and to reduce redundancies.

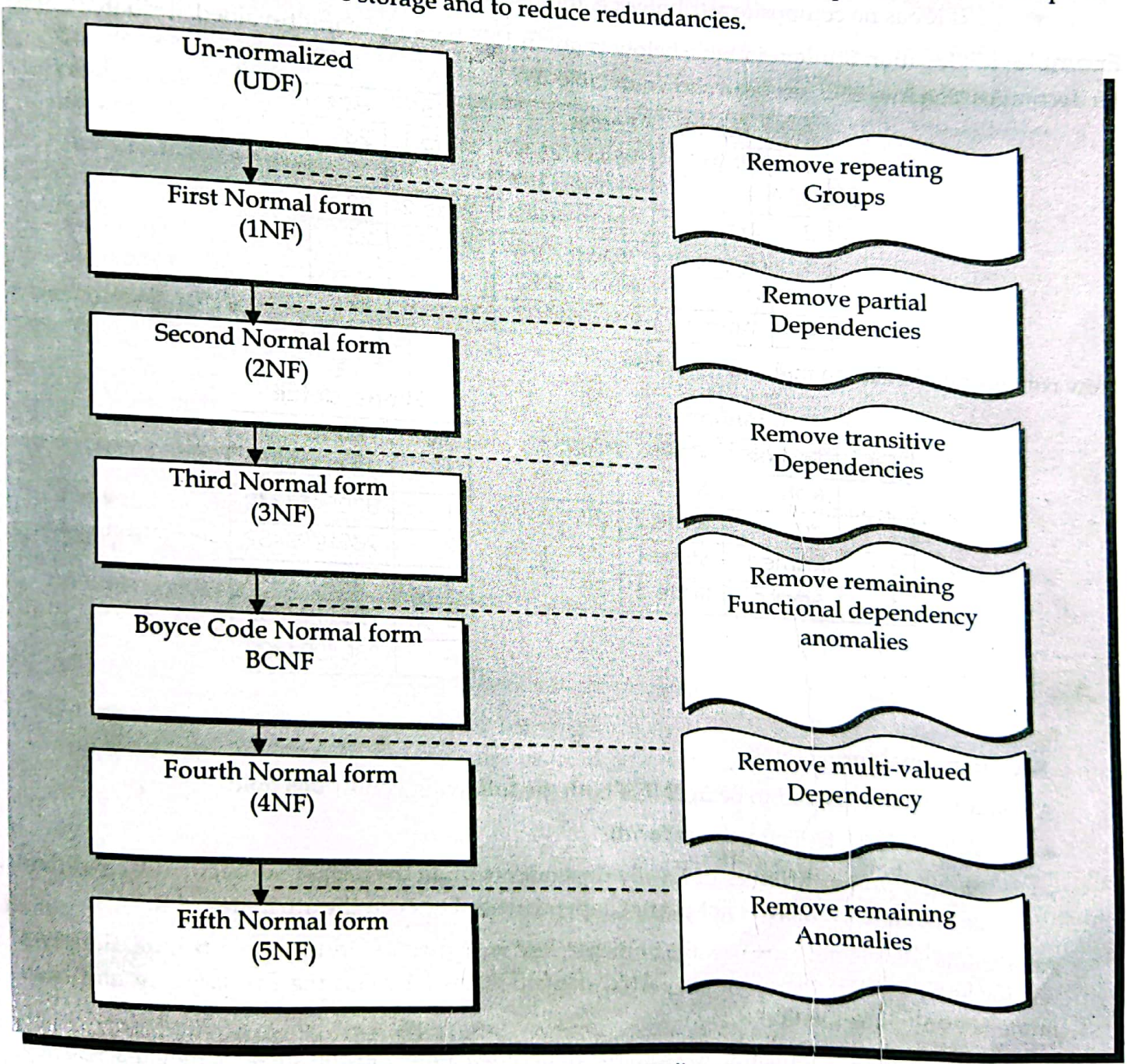


Figure 4.1: Types of Normalization

Normal Forms Based on Primary Keys

The normal forms first, second and third are called primary key based normal forms. Here each of the non-prime attributes of the relation depends upon the prime attribute of the relation.

a. First Normal Form

If a relation contain composite or multi-valued attribute, it violates first normal form or a relation is in first normal form if it does not contain any composite or multi-valued attribute.

Simply, A relation R is in 1 NF if it holds following two properties;

- If R has no multivalued attributes
- If R has no composite attributes i.e. it must have simple attributes

Example 4.7: Relation **Student** in table below is not in 1NF because of multi-valued attribute Phone. Its decomposition into 1NF has been shown in table 2.

Sid	Sname	Phone	State	Age
1	Ram	9848372632, 8905454342	State 1	23
2	Hari	9851093482	State 2	33
3	Ramesh	9848372098 9851123032	State 1	22
4	Anand	9804094302	State 3	29

Now convert given relation into 1NF as below,

Student

Sid	Sname	State	Age
1	Ram	State 1	23
2	Hari	State 2	33
3	Ramesh	State 1	22
4	Anand	State 3	29

Phone_detail

Sid	Phone
1	9848372632
1	8905454342
2	9851093482
3	9848372098
3	9851123032
4	9804094302

b. Second normal form (2NF)

A relation R (table) is said to be in 2NF if both the following conditions hold:

- Table is in 1NF (First normal form)
- No non-prime attributes of R fully dependents upon the proper subset of any candidate key of table i.e. there is not partial dependencies between the attributes of R.

An attribute that is not part of any candidate key is known as non-prime attribute. Simply a relation is in 2 NF if there are no partial dependencies between the primary key and non-prime keys of given relation R.

Example 4.8: Suppose a school wants to store the data of teachers and the subjects they teach. They create a table that looks like this: Since a teacher can teach more than one subjects, the table can have multiple rows for a same teacher.

Teacher

Tid	Tsubject	Tage
111	Maths	38
111	Physics	38
222	Biology	38
333	Physics	40
333	Chemistry	40

Candidate Keys: {Tid, Tsubject}

Non prime attribute: {Tage}

The table is in 1 NF because each attribute has atomic values. However, it is not in 2NF because non prime attribute Tage is dependent on Tid alone which is a proper subset of candidate key. This violates the rule for 2NF as the rule says "no non-prime attribute is dependent on the proper subset of any candidate key of the table".

To make the table complies with 2NF we can break it in two tables like this:

teacher_details

Tid	Tage
111	38
222	38
333	40

teacher_subject

Tid	Tsubject
111	Maths
111	Physics
222	Biology
333	Physics
333	Chemistry

Now the tables comply with Second normal form (2NF).

c. **Third Normal form (3NF)**

A table design is said to be in 3NF if both the following conditions hold:

- Table must be in 2NF
- No transitive functional dependency between the attributes of given table.

In other words 3NF can be explained like this: A table is in 3NF if it is in 2NF and for each functional dependency X→Y at least one of the following conditions hold:

- X is a super key of table
- Y is a prime attribute of table

Example 4.9: Suppose a company wants to store the complete address of each employee, they create a table named employee_details that looks like this:

emp_id	emp_name	emp_zip	emp_state	emp_city	emp_district
1001	John	282005	State 1	Dhankuta	Jhapa
1002	Ajeet	222008	State 2	Janakpur	Sarlahi
1006	Lora	282007	State 2	Janakpur	Sarlahi
1101	Lilly	292008	State 3	Kathmandu	Ktm
1201	Steve	222999	State 7	Dhangadhi	Kailali

Super keys: {emp_id}, {emp_id, emp_name}, {emp_id, emp_name, emp_zip}...so on
Candidate Keys: {emp_id}

Non-prime attributes: all attributes except emp_id are non-prime as they are not part of any candidate keys.

Here, emp_state, emp_city & emp_district dependent on emp_zip. And, emp_zip is dependent on emp_id that makes non-prime attributes (emp_state, emp_city & emp_district) transitively dependent on super key (emp_id). This violates the rule of 3NF. To make this table complies with 3NF we have to break the table into two tables to remove the transitive dependency:

Employee

emp_id	emp_name	emp_zip
1001	John	282005
1002	Ajeet	222008
1006	Lora	282007
1101	Lilly	292008
1201	Steve	222999

Employee_zip

emp_zip	emp_state	emp_city	emp_district
282005	State 1	Dhankuta	Jhapa
222008	State 2	Janakpur	Sarlahi
282007	State 2	Janakpur	Sarlahi
292008	State 3	Kathmandu	Ktm
222999	State 7	Dhangadhi	Kailali

d. Boyce Codd normal form (BCNF)

It is an advance version of 3NF that's why it is also referred as 3.5NF. BCNF is stricter than 3NF. A table complies with BCNF if it is in 3NF and for every functional dependency X→Y, X should be the super key of the table.

Mathematically,

A relation R is said to be in BCNF with respect to all the functional dependencies of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, if at least one of the following holds:

- $\alpha \rightarrow \beta$ is a trivial functional dependency (that is, $\beta \subseteq \alpha$).
- α is a candidate key for schema R.

Example 4.10: Suppose there is a company wherein employees work in more than one department. They store the data like this:

emp_id	emp_nationality	emp_dept	dept_type	dept_no_of_emp
1001	Nepalese	Production and planning	D001	200
1001	Nepalese	Store department	D001	250
1002	Indian	Technical support	D134	100
1002	Indian	Purchasing department	D134	600

Functional dependencies in the above table are:

emp_id→emp_nationality

emp_dept→{dept_type, dept_no_of_emp}

Candidate key: {emp_id, emp_dept}

The table is not in BCNF as neither emp_id nor emp_dept alone are keys.

To make the table comply with BCNF we can break the table in three tables like this:

emp_nationality

emp_id	emp_nationality
1001	Nepalese
1002	Indian

Table 1

emp_dept_maping

emp_id	emp_dept
1001	Production and planning
1001	Store department
1002	Technical support
1002	Purchasing department

Table 3

emp_dept

emp_dept	dept_type	dept_no_of_emp
Production and planning	D001	200
Store department	D001	250
Technical support	D134	100
Purchasing department	D134	600

Table 2

Functional dependencies:

emp_id → emp_nationality

emp_dept → {dept_type, dept_no_of_emp}

Candidate keys:

For first table: emp_id

For second table: emp_dept

For third table: {emp_id, emp_dept}

This is now in BCNF as in both the functional dependencies left side part is a key.

Differences between 3NF and BCNF

3NF	BCNF
A table or a relation is considered to be in third normal form only if the table is already in 2NF and there is no non-prime attribute transitively dependent on the candidate key of a relation.	BCNF is considered to be the stronger than 3NF. The relation R to be in BCNF must be in 3NF. And wherever a non-trivial functional dependency A→B holds in relation R, then A must be a super key of relation R.
No non-prime attribute must be transitively dependent on the Candidate key.	For any trivial dependency in a relation R say X→Y, X should be a super key of relation R.
3NF can be obtained without sacrificing all dependencies.	Dependencies may not be preserved in BCNF.
Lossless decomposition can be achieved in 3NF.	Lossless decomposition is hard to achieve in BCNF.

e. Fourth Normal Form

A table R is in fourth normal form (4NF) if and only if it satisfied following two conditions simultaneously:

- R is already in BCNF or in 3NF
- If it contains no multi-valued attributes

Equivalently, we can that a relation schema R is in 4NF if, for every nontrivial multi-valued dependency X →→Y, X is a super key for R. The goal of fourth normal form is to eliminate nontrivial multi-valued dependencies from a table by projecting them onto separate smaller tables, thus eliminating the update anomalies associated with the multi-valued dependencies.

Example 4.11: Consider the following relation subject which has the attributes course, instructor, and textbook.

Subject (course, Instructor, Text-book)

Course	Instructor	Text-Book
CS1	Indra	DBMS
CS1	Bhupi	DBMS
CS2	Arjun	NM
CS2	Bhupendra	NM
CS3	Indra	CG
CS3	Bhupi	CG
CS3	Arjun	CG

In this relation for each course there are many instructors and also for each course there are many text-books. But instructor and text-book are independent of each other. Thus, the given relation Subject is not in fourth normal form because of multi-valued dependency between attributes.

Course →→ instructor

Course →→ textbooks

The relation subject can be converted to fourth normal form by splitting the relation Subject into two relations Teacher and Text as below

Teacher (Course, Instructor)

Text (Course, Textbook)

Example 4.12: Consider the relation Employee with the attributes employee number, project name, and department name as shown below:

Employee (Eno, Pname, Dname)

Where, Eno stands for Employee number, Pname for project name, and Dname for department name. The relation Employee has the following multi-valued dependencies:

Eno →→ Pname

Eno →→ Dname

Since, Eno is not the super key of the relation Employee so it is not in 4NF. To convert the relation to 4 NF, we can decompose Employee relation into two relations Emp-Proj and Emp- Dept as shown below.

Emp-proj (Eno, Pname)

Emp-dept (Eno, Dname)

WHAT IS DECOMPOSITION?

Decomposition is the process of breaking down in parts or elements. It replaces a relation with a collection of smaller relations. It breaks the table into multiple tables in a database. It should always be lossless, because it confirms that the information in the original relation can be accurately reconstructed based on the decomposed relations. If there is no proper decomposition of the relation, then it may lead to problems like loss of information.

Properties of Relational Decomposition

Following are the properties of Decomposition,

1. Lossless Decomposition
2. Dependency Preservation
3. Lack of Data Redundancy

Lossless Decomposition

Decomposition must be lossless. It means that the information should not get lost from the relation that is decomposed. It gives a guarantee that the join will result in the same relation as it was decomposed.

Example 4.13: Let's take 'E' is the Relational Schema, with instance 'e'; is decomposed into: E1, E2, E3 . . . En; with instance: e1, e2, e3 . . . en, If e1 ⋈ e2 ⋈ e3 . . . . ⋈ en, then it is called as 'Lossless Join Decomposition'.

In the above example, it means that, if natural joins of all the decomposition give the original relation, then it is said to be lossless join decomposition.

Example 4.14: Employee_Department

Employee_Depatment

Eid	Ename	Age	City	Salary	Deptid	DeptName
E001	ABC	29	Ktm	20000	D001	Finance
E002	PQR	30	Ktm	30000	D002	Production
E003	LMN	25	Banepa	5000	D003	Sales
E004	XYZ	24	Banepa	4000	D004	Marketing
E005	STU	32	Pokhara	25000	D005	Human Resource

Decompose the above relation into two relations to check whether decomposition is lossless or lossy.

Now, we have decomposed the relation that is Employee and Department.

Relation 1: Employee

Employee

Eid	Ename	Age	City	Salary
E001	ABC	29	Pune	20000
E002	PQR	30	Pune	30000
E003	LMN	25	Mumbai	5000
E004	XYZ	24	Mumbai	4000
E005	STU	32	Bangalore	25000

Employee Schema contains (Eid, Ename, Age, City, Salary)

Relation 2: Department

Department

Deptid	Eid	DeptName
D001	E001	Finance
D002	E002	Production
D003	E003	Sales
D004	E004	Marketing
D005	E005	Human Resource

- Department Schema contains (Deptid, Eid, DeptName).
- So, the above decomposition is a Lossless Join Decomposition, because the two relations contain one common field that is 'Eid' and therefore join is possible.
- Now apply natural join on the decomposed relations.

Employee ⋈ Department

Eid	Ename	Age	City	Salary	Deptid	DeptName
E001	ABC	29	Pune	20000	D001	Finance
E002	PQR	30	Pune	30000	D002	Production
E003	LMN	25	Mumbai	5000	D003	Sales
E004	XYZ	24	Mumbai	4000	D004	Marketing
E005	STU	32	Bangalore	25000	D005	Human Resource

Hence, the decomposition is Lossless Join Decomposition.

If the Employee table contains (Eid, Ename, Age, City, Salary) and Department table contains (Deptid and DeptName), then it is not possible to join the two tables or relations, because there is no common column between them. And it becomes Lossy Join Decomposition.

Dependency Preservation

- Dependency is an important constraint on the database.
- Every dependency must be satisfied by at least one decomposed table.
- If {A → B} holds, then two sets are functional dependent. And, it becomes more useful for checking the dependency easily if both sets in a same relation.
- This decomposition property can only be done by maintaining the functional dependency.
- In this property, it allows to check the updates without computing the natural join of the database structure.

Lack of Data Redundancy

- Lack of Data Redundancy is also known as a Repetition of Information.
- The proper decomposition should not suffer from any data redundancy.
- The careless decomposition may cause a problem with the data.
- The lack of data redundancy property may be achieved by Normalization process.

FUNCTIONAL DEPENDENCY INFERENCE RULES (ARMSTRONG'S AXIOMS)

a. Reflexivity

If $Y \subseteq X$ then $X \rightarrow Y$. The axiom of reflexivity indicates that given a set of attributes the set itself functionally determines any of its own subsets.

b. Augmentation

If $X \rightarrow Y$ and Z is a subset of table R (i.e., Z is any set of attributes in R), then $XZ \rightarrow YZ$. The axiom of augmentation indicates that we can augment the left side of the functional dependency or both sides conveniently with one or more attributes. The axiom does not allow augmenting the right-hand side alone. The augmentation rule can be represented as If $X \rightarrow Y$ then $XZ \rightarrow Y$

c. Transitivity

If $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$. The axiom of transitivity indicates that if one attribute uniquely determines a second attribute and this, in turn, uniquely determines a third one, then the first attributes determines the third one. Consider three parallel lines X, Y, and Z. The line X is parallel to line Y. The line Y is parallel to line Z then it implies that line X is parallel to line Z. This property is called transitivity.

d. Pseudo transitivity

If $X \rightarrow Y$ and $YW \rightarrow Z$ then $XW \rightarrow Z$. Transitivity is a special case of pseudo transitivity when W is null. The axiom of pseudo transitivity is a generalization of the transitivity axiom.

e. Union

If $X \rightarrow Y$ and $X \rightarrow Z$ then $X \rightarrow YZ$. The axiom of union indicates that if there are two functional dependencies with the same determinant it is possible to form a new functional dependency that preserves the determinant and has its right-hand side the union of the right-hand sides of the two functional dependencies. The union rule can be illustrated with the example of PINCODE. The PINCODE is used to identify city as well as PINCODE is used to identify state. This implies that PINCODE determines both city and state.

f. Decomposition

If $X \rightarrow YZ$ then $X \rightarrow Y$ and $X \rightarrow Z$. The axiom of decomposition indicates that the determinant of any functional dependency can uniquely determine any individual attribute or any combination of attributes of the right-hand side of the functional dependency. The decomposition can be illustrated with an example of Book ID. The Book ID determines the title and the author similar to $(X \rightarrow YZ)$ which implies that Book ID determines title $(X \rightarrow Y)$ and Book ID determines Author $(X \rightarrow Z)$

CLOSURE OF SET OF FUNCTIONAL DEPENDENCIES

The Closure of Functional Dependency means the complete set of all possible attributes that can be functionally derived from given functional dependency using the inference rules known as Armstrong's Rules. If "F" is a functional dependency then closure of functional dependency can be denoted using "{F}+". There are three steps to calculate closure of functional dependency. These are:

- **Step-1:** Add the attributes which are present on Left Hand Side in the original functional dependency.
- **Step-2:** Now, add the attributes present on the Right Hand Side of the functional dependency.
- **Step-3:** With the help of attributes present on Right Hand Side, check the other attributes that can be derived from the other given functional dependencies. Repeat this process until all the possible attributes which can be derived are added in the closure.

Algorithm: Determining X+, the closure of X under F.

Input: Let F be a set of FDs for relation R.

Steps:

1. X+ = X //initialize X+ to X
2. for each FD: Y → Z in F Do
   If Y ⊆ X+ Then //If Y is contained in X+
   X+ = X+ ∪ Z //add Z to X+
   End If
   End For
3. Return X+ //Return closure of X

Output: Closure X+ of X under F

Example 4.15: Let R = (A, B, C, D)

F= {A→B, A→C, BC→D}. List several member of F+

Solution:

Since A→B, A→C then A→BC [by union rule]

Since A→BC, BC→D then A→D [by transitive rule]

Since A→C, BC→D then AB→D [by pseudo transitivity rule]

Hence F+ = {A→B, A→C, BC→D, A→BC, A→D, AB→D}

Example 4.16: Let R = (A, B, C, G, H, I)

Given F= {A→B, A→C, CG→H, CG→I, B→H}, Compute closure of F (F+).

Solution:

Since A→B, B→H then A→H [by transitivity rule]

Since A→B, A→C then A→BC [by union rule]

Since A→C then AG→CG [by augmentation rule]

Since AG→CG, CG→I then AG→I [by transitivity rule]

Since CG →H, CG→I then CG→HI [by union rule]

Hence, F+ = {A→A, B→B, C→C, H→H, G→G, I→I, A→B, A→C, CG→H, G→I, CG→HI, B→H, A→H, AG→I, CG→HI}

Here, first six FDs obtain by reflexive axiom.

Example 4.17: Let R = (A, B, C, D) and F= {A→B, A→C, BC→D} then compute F+.

Solution:

Since A→B and A→C then by union rule A→BC.

Since BC →D, then by decomposition B→D, C→D. Again by transitivity A→B &B →D ⇒ A→D and A→C and C→D ⇒ A→D.

Hence, F+ = {A→A, B→B, C→C, D→D, A→B, A→C, BC→D, B→D, C→D, A→D}

CLOSURE OF SETS ATTRIBUTES WITH FUNCTIONAL DEPENDENCY

The set of all those attributes which can be functionally determined from an attribute set is called as a closure of that attribute set. Closure of attribute set {X} is denoted as {X}+.

Steps to Find Closure of an Attribute Set

Following steps are followed to find the closure of an attribute set-

Step 1: Add the attributes contained in the attribute set for which closure is being calculated to the result set.

Step 2: Recursively add the attributes to the result set which can be functionally determined from the attributes already contained in the result set.

Example 4.18: Consider a relation R (A, B, C, D, E, F, G) with the functional dependencies

$A \rightarrow BC$

$BC \rightarrow DE$

$D \rightarrow F$

$CF \rightarrow G$

Now, let us find the closure of some attributes and attribute sets-

Closure of attribute A:

$A+ = \{A\}$

$= \{A, B, C\}$ (Using $A \rightarrow BC$)

$= \{A, B, C, D, E\}$ (Using $BC \rightarrow DE$)

$= \{A, B, C, D, E, F\}$ (Using $D \rightarrow F$)

$= \{A, B, C, D, E, F, G\}$ (Using $CF \rightarrow G$)

Thus,

$A+ = \{A, B, C, D, E, F, G\}$

Closure of attribute D:

$D+ = \{D\}$

$= \{D, F\}$ (Using $D \rightarrow F$)

We cannot determine any other attribute using attributes D and F contained in the result set.

Thus,

$D+ = \{D, F\}$

Closure of attribute set {B, C}:

$\{B, C\}+ = \{B, C\}$

$= \{B, C, D, E\}$ (Using $BC \rightarrow DE$)

$= \{B, C, D, E, F\}$ (Using $D \rightarrow F$)

$= \{B, C, D, E, F, G\}$ (Using $CF \rightarrow G$)

Thus,

$\{B, C\}+ = \{B, C, D, E, F, G\}$

Example 4.19: Let R = {A, B, C, D, E} and F= {AB→C, A→D, D→E, AC→B} then compute (AB) +

Solution:

Result = {AB}

AB→C hence result = {ABC}

A→D hence result = {ABCD}

D→E hence result = {ABCDE}

AC→B [B is already in the result]

So result= {ABCDE}

Compute result for {A} + for above example:

Result = {A}

AB→C [no change] result = {A}

A→D hence result = {AD}

D→E hence result = {ADE}

AC→B [no change] result = {ADE}

So result= {ADE}

Feature of Good Relational Design

It is possible to generate a set of relation schemas directly from the E-R design. Obviously, the goodness (or badness) of the resulting set of schemas depends on how good the E-R design was in the first place. Later in this chapter, we shall study precise ways of assessing the desirability of a collection of relation schemas. However, we can go a long way toward a good design using concepts we have already studied.

- Larger Schemas
- **Smaller Schemas**
- Remove redundancy
- Anomalies
- Decomposition
- Lossless decomposition
- Normalization

Denormalization for performance

Occasionally database designers choose a schemathat has redundant information; that is, it is not normalized. They use the redundancy to improve performance for specific applications. The penalty paid for not using a normalized schema is the extra work (in terms of coding time and execution time) to keep redundant data consistent.
For instance, suppose all course prerequisites have to be displayed along with a course information, every time a course is accessed. In our normalized schema, this requires a join of *course* with *prereq*.
One alternative to computing the join on the fly is to store a relation containing all the attributes of *course* and *prereq*. This makes displaying the "full" course information faster. However, the information for a course is repeated for every course prerequisite, and all copiesmust be updated by the application, whenever a course prerequisite is added or dropped. The process of taking a normalized schema and making it nonnormalized is called **denormalization**, and designers use it to tune performance of systems to support time-critical operations.
A better alternative, supported by many database systems today, is to use the normalized schema, and additionally store the join of *course* and *prereq* as a materialized view. (Recall that amaterialized view is a viewwhose result is stored in the database and brought up to date when the relations used in the view are updated.) Like denormalization, using materialized views does have space and time overhead; however, it has the advantage that keeping the view up to date is the job of the database system, not the application programmer.

// **Multivalued Dependencies Read**

Chapter 6: Integrity Constraints

- Domain Constraints
- Referential Integrity
- Assertions
- Triggers
- Functional Dependencies

Domain Constraints

- Integrity constraints guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency.
- Domain constraints are the most elementary form of integrity constraint.
- They test values inserted in the database, and test queries to ensure that the comparisons make sense.

Domain Constraints (Cont.)

- The **check** clause in SQL-92 permits domains to be restricted:
  - Use **check** clause to ensure that an hourly-wage domain allows only values greater than a specified value.

    create domain *hourly-wage* **numeric(5,2)**
    constraint *value-test* **check**(*value* >= 4.00)
  - The domain *hourly-wage* is declared to be a decimal number with 5 digits, 2 of which are after the decimal point
  - The domain has a constraint that ensures that the hourly-wage is greater than 4.00.
  - The clause **constraint** *value-test* is optional; useful to indicate which constraint an update violated.

Referential Integrity

- Ensures that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation.
  - Example: if "Perryridge" is a branch name appearing in one of the tuples in the *account* relation, then there exists a tuple in the *branch* relation for branch "Perryridge".
- Formal Definition
  - Let $r_1(R_1)$ and $r_2(R_2)$ be relations with primary keys K_1 and K_2 respectively.
  - The subset α of R_2 is a *foreign key* referencing K_1 in relation r_1, if for every t_2 in r_2 there must be a tuple t_1 in r_1 such that $t_1[K_1] = t_2[\alpha]$.
  - Referential integrity constraint: $\Pi_{\alpha} (r_2) \subseteq \Pi_{K_1} (r_1)$

Referential Integrity in the E-R Model

- Consider relationship set R between entity sets E_1 and E_2. The relational schema for R includes the primary keys K_1 of E_1 and K_2 of E_2.

  Then K_1 and K_2 form foreign keys on the relational schemas for E_1 and E_2 respectively.

- Weak entity sets are also a source of referential integrity constraints. For, the relation schema for a weak entity set must include the primary key of the entity set on which it depends.

Database Modification

- The following tests must be made in order to preserve the following referential integrity constraint:

$$\Pi_{\alpha}\ (r_2)\ \subseteq\ \Pi_{K}\ (r_1)$$

- **Insert.** If a tuple t_2 is inserted into r_2, the system must ensure that there is a tuple t_1 in r_1 such that $t_1[K] = t_2[\alpha]$. That is

$$t_2[\alpha]\ \in\ \Pi_{K}\ (r_1)$$

- **Delete.** If a tuple t_1 is deleted from r_1, the system must compute the set of tuples in r_2 that reference t_1:

$$\sigma_{\alpha\ =\ t_1[K]}\ (r_2)$$

  If this set is not empty, either the delete command is rejected as an error, or the tuples that reference t_1 must themselves be deleted (cascading deletions are possible).

Database Modification (Cont.)

- **Update.** There are two cases:
  - If a tuple t_2 is updated in relation r_2 and the update modifies values for the foreign key α, then a test similar to the insert case is made. Let t_2' denote the new value of tuple t_2. The system must ensure that

$$t_2'[\alpha] \in \Pi_K (r_1)$$

  - If a tuple t_1 is updated in r_1, and the update modifies values for the primary key (K), then a test similar to the delete case is made. The system must compute

$$\sigma_{\alpha = t_1[K]} (r_2)$$

    using the old value of t_1 (the value before the update is applied). If this set is not empty, the update may be rejected as an error, or the update may be cascaded to the tuples in the set, or the tuples in the set may be deleted.

Referential Integrity in SQL

- Primary and candidate keys and foreign keys can be specified as part of the SQL **create table** statement:
  - The **primary key** clause of the **create table** statement includes a list of the attributes that comprise the primary key.
  - The **unique key** clause of the **create table** statement includes a list of the attributes that comprise a candidate key.
  - The **foreign key** clause of the **create table** statement includes both a list of the attributes that comprise the foreign key and the name of the relation referenced by the foreign key.

Referential Integrity in SQL - Example

create table *customer*
(*customer-name* char(20) **not null**,
customer-street char(30),
customer-city char(30),
primary key (*customer-name*))

create table *branch*
(*branch-name* char(15) **not null**,
branch-city char(30),
assets integer,
primary key (*branch-name*))

Referential Integrity in SQL - Example (Cont.)

create table *account*
(*branch-name* char(15),
account-number char(10) **not null**,
balance integer,
primary key (*account-number*),
foreign key (*branch-name*) **references** *branch*)

create table *depositor*
(*customer-name* char(20) **not null**,
account-number char(10) **not null**,
primary key (*customer-name*, *account-number*),
foreign key (*account-number*) **references** *account*,
foreign key (*customer-name*) **references** *customer*)

Cascading Actions in SQL

create table *account*

. . .

foreign key (*branch-name*) **references** *branch*

on delete cascade

on update cascade,

. . .)

- Due to the **on delete cascade** clauses, if a delete of a tuple in *branch* results in referential-integrity constraint violation, the delete “cascades” to the *account* relation, deleting the tuple that refers to the branch that was deleted.
- Cascading updates are similar.

Cascading Actions in SQL (Cont.)

- If there is a chain of foreign-key dependencies across multiple relations, with **on delete cascade** specified for each dependency, a deletion or update at one end of the chain can propagate across the entire chain.
- If a cascading update or delete causes a constraint violation that cannot be handled by a further cascading operation, the system aborts the transaction. As a result, all the changes caused by the transaction and its cascading actions are undone.

Assertions

- An *assertion* is a predicate expressing a condition that we wish the database always to satisfy.
- An assertion in SQL-92 takes the form

  create assertion <assertion-name> **check** <predicate>

- When an assertion is made, the system tests it for validity. This testing may introduce a significant amount of overhead; hence assertions should be used with great care.

Assertion Example

- The sum of all loan amounts for each branch must be less than the sum of all account balances at the branch.

create assertion *sum-constraint* **check**
(**not exists** (**select** * **from** *branch*
where (**select sum**(*amount*) **from** *loan*
where *loan.branch-name* = *branch.branch-name*)
>= (**select sum**(*amount*) **from** *account*
where *loan.branch-name* = *branch.branch-name*)))

Assertion Example

- Every loan has at least one borrower who maintains an account with a minimum balance of $1000.00.

create assertion *balance-constraint* **check**
(**not exists** (**select** * **from** *loan*
where not exists (**select** *
from *borrower*, *depositor*, *account*
where *loan.loan-number = borrower.loan-number*
and *borrower.customer-name = depositor.customer-name*
and *depositor.account-number = account.account-number*
and *account.balance* >= 1000)))

Triggers

- A *trigger* is a statement that is executed automatically by the system as a side effect of a modification to the database.
- To design a trigger mechanism, we must:
  - Specify the conditions under which the trigger is to be executed.
  - Specify the actions to be taken when the trigger executes.
- The SQL-92 standard does not include triggers, but many implementations support triggers.

Trigger Example

- Suppose that instead of allowing negative account balances, the bank deals with overdrafts by
  - setting the account balance to zero
  - creating a loan in the amount of the overdraft
  - giving this loan a loan number identical to the account number of the overdrawn account
- The condition for executing the trigger is an update to the *account* relation that results in a negative *balance* value.

Trigger Example (Cont.)

define trigger *overdraft* **on update of** *account T*
(**if new** *T.balance* $<$ 0
then (**insert into** *loan* **values**
(*T.branch-name, T.account-number*, $-$ **new** *T.balance*)
insert into *borrower*
(**select** *customer-name, account-number*
from *depositor*
where *T.account-number = depositor.account-number*)
update *account S*
set *S.balance* = 0
where *S.account-number = T.account-number*))

The keyword **new** used before *T.balance* indicates that the value of *T.balance* after the update should be used; if it is omitted, the value before the update is used.

Functional Dependencies

- Constraints on the set of legal relations.
- Require that the value for a certain set of attributes determines uniquely the value for another set of attributes.
- A functional dependency is a generalization of the notion of a *key*.

Functional Dependencies (Cont.)

- Let R be a relation schema

$$\alpha \subseteq R,\ \beta \subseteq R$$

- The functional dependency

$$\alpha \rightarrow \beta$$

  holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α, they also agree on the attributes β. That is,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- K is a superkey for relation schema R if and only if $K \rightarrow R$
- K is a candidate key for R if and only if
  - $K \rightarrow R$, and
  - for no $\alpha \subset K$, $\alpha \rightarrow R$

Functional Dependencies (Cont.)

- Functional dependencies allow us to express constraints that cannot be expressed using superkeys. Consider the schema:

  Loan-info-schema = (*branch-name, loan-number, customer-name, amount*).

  We expect this set of functional dependencies to hold:

  $$\textit{loan-number} \rightarrow \textit{amount}$$
  $$\textit{loan-number} \rightarrow \textit{branch-name}$$

  but would not expect the following to hold:

  $$\textit{loan-number} \rightarrow \textit{customer-name}$$

Use of Functional Dependencies

- We use functional dependencies to:
  - test relations to see if they are legal under a given set of functional dependencies. If a relation r is legal under a set F of functional dependencies, we say that *r satisfies F*.
  - specify constraints on the set of legal relations; we say that *F holds* on R if all legal relations on R satisfy the set of functional dependencies F.
- Note: A specific instance of a relation schema may satisfy a functional dependency even if the functional dependency does not hold on all legal instances. For example, a specific instance of *Loan-schema* may, by chance, satisfy *loan-number* $\rightarrow$ *customer-name*.

Closure of a Set of Functional Dependencies

- Given a set *F* set of functional dependencies, there are certain other functional dependencies that are logically implied by *F*.
- The set of all functional dependencies logically implied by *F* is the *closure* of *F*.
- We denote the *closure* of *F* by F^+.
- We can find all of F^+ by applying Armstrong's Axioms:
  - if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$ (**reflexivity**)
  - if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$ (**augmentation**)
  - if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$ (**transitivity**)

  These rules are sound and complete.

Closure (Cont.)

- We can further simplify computation of F^+ by using the following additional rules.
  - If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds (**union**)
  - If $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds (**decomposition**)
  - If $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds (**pseudotransitivity**)

  The above rules can be inferred from Armstrong's axioms.

Example

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B$
  $A \rightarrow C$
  $CG \rightarrow H$
  $CG \rightarrow I$
  $B \rightarrow H\}$
- some members of F^+
  - $A \rightarrow H$
  - $AG \rightarrow I$
  - $CG \rightarrow HI$

Closure of Attribute Sets

- Define the *closure* of α under *F* (denoted by α^+) as the set of attributes that are functionally determined by α under *F*:

$$\alpha \rightarrow \beta \text{ is in } F^+ \Leftrightarrow \beta \subseteq \alpha^+$$

- Algorithm to compute α^+, the closure of α under *F*

```
result := α;
while (changes to result) do
        for each β → γ in F do
            begin
                if β ⊆ result then result := result ∪ γ;
            end
```

Example

- $R = (A, B, C, G, H, I)$

  $F = \{A \rightarrow B$
  $A \rightarrow C$
  $CG \rightarrow H$
  $CG \rightarrow I$
  $B \rightarrow H\}$

- (AG^+)
  1. $result = AG$
  2. $result = ABCG$ ($A \rightarrow C$ and $A \subseteq AGB$)
  3. $result = ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
  4. $result = ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)

- Is AG a candidate key?
  1. $AG \rightarrow R$
  2. does $A^+ \rightarrow R$?
  3. does $G^+ \rightarrow R$?

Canonical Cover

- Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F.
  - Attribute A is extraneous in α if $A \in \alpha$ and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
  - Attribute A is extraneous in β if $A \in \beta$ and the set of functional dependencies $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F.
- A *canonical cover* F_c for F is a set of dependencies such that F logically implies all dependencies in F_c and F_c logically implies all dependencies in F, and further
  - No functional dependency in F_c contains an extraneous attribute.
  - Each left side of a functional dependency in F_c is unique.

Canonical Cover (Cont.)

- Compute a canonical cover for *F*:

  repeat

    Use the union rule to replace any dependencies in *F*

      $\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1 \beta_2$

    Find a functional dependency $\alpha \rightarrow \beta$ with an

      extraneous attribute either in α or in β

    If an extraneous attribute is found, delete it from $\alpha \rightarrow \beta$

  until *F* does not change

Example of Computing a Canonical Cover

- $R = (A, B, C)$
  $F = \{A \rightarrow BC$
  $B \rightarrow C$
  $A \rightarrow B$
  $AB \rightarrow C\}$
- Combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$
- A is extraneous in $AB \rightarrow C$ because $B \rightarrow C$ logically implies $AB \rightarrow C$.
- C is extraneous in $A \rightarrow BC$ since $A \rightarrow BC$ is logically implied by $A \rightarrow B$ and $B \rightarrow C$.
- The canonical cover is:

  $A \rightarrow B$
  $B \rightarrow C$